

Dandelion Hashable: Beyond Billion Memory Requests Per Second on a Commodity Server

Antonios Katsarakis Vasilis Gavrielatos Nikos Ntarmos
Huawei Research

In High-Performance Parallel and Distributed Computing (HPDC'24)

Concurrent Growing Hashtables

redis ZooKeeper M TIKV

- Ubiquitous in todays cloud
- Offer *Get, Put, Insert, Delete* and index *Resizes* to store and access KV pairs

State-of-the-art

- Keep data **in-memory**
- Ensure **strong consistency**
- Exploit **concurrency** to cope with **high throughput** needs of modern services

Deficiencies of Existing Work

Strongly consistent but **cannot maximize throughput** in memory-resident workloads

- Block excessively
- Handle memory accesses inefficiently

Concurrent but **practically blocking**

Prior art may offer non-blocking Gets but compromise *Deletes/Inserts/Resizes*

- Inserts leading to Resize block every request until all objects copied to larger index
- Open-addressing designs: slow Deletes that block all requests to free index slots

In-memory but **not memory aware**

- Multiple memory accesses even for small (16B) KVs, no collision/resizes
- No overlapped memory accesses: stall each Get,Insert,Delete waiting for data from memory
- Bad occupancy: few index slots filled before resize → harms memory/performance

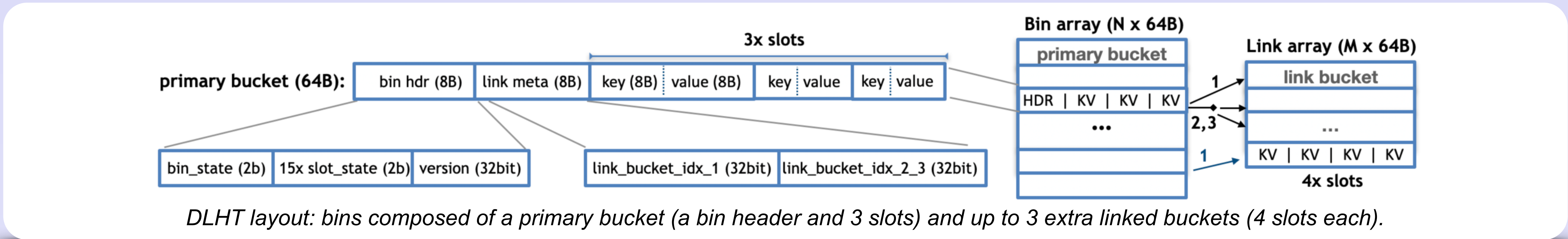
	Collision handling	Practically non-blocking operations					Memory access awareness		
		Gets	Puts	Inserts	Deletes (that free index slots)	Resizes (non-blocking: safe Get,...,Del during resize)	Overlaps memory accesses (i.e., exploits s/w prefetch)	Minimizes memory traffic (e.g., supports index inlining)	Occupancy until resize (from § 5.1.5 with wyhash)
GrowT	open-addressing	✓	✓	✓	Blocking (until all index is transferred)	Parallel, Blocking	✗	✓	30-50%
Folly	open-addressing	✓	✓	✓	✗	✗	✗	✓	—
DRAMHIT	open-addressing	✓	Only upserts	✗	✗	✗	✓ Reorders requests	✓	—
MICA	closed-addressing	✓	Blocking	Blocking	✗	✗	✓	✗	—
CLHT	closed-addressing	Unique values	✗	✓	✓	Serial, Blocking	✗	✓	1-5%
DLHT	closed-addressing	✓	✓	✓	✓	✓	✓	✓	61-72%

Key features for memory-resident performance across state-of-the-art concurrent in-memory hashtables and DLHT.

DLHT to the rescue!!

No open-addressing = no severe blocking on deletes

Get, Insert, Delete via extending non-blocking algorithms of CLHT based on **8B header (per bin)** for synchronization.



+ **Memory compact design** of bin header

- includes *bin state* for practically non-blocking resizes
- states* for up to 15 slots to improve occupancy over lock-free CLHT

+ **Batched API** pipelined processing

- Software prefetching to overlap memory accesses
- Guaranteed in-order request completion

+ **Bounded cacheline chaining**

Link array: 4-8x smaller than Bin array (non-blocking resize if full)

Most bins = 1 bucket → Most requests = 1 memory access

